

128 South Goran
Dhaka, Dhaka-1219
(+880) 1402120931
Email: kazisahib5@gmail.com
Github: [ksahib \(Kazi Sahib\)](#)

Kazi Sahib

SKILLS

Languages:

C/C++, Fortran, ProLog, Python, Java, Dart, PHP, JavaScript, MySQL, HTML, CSS, LaTeX

Frameworks:

Parallel And HPC:

OpenMP, pthreads

Distributed And Backend:

gRPC, Protocol Buffers, Drogon, FastAPI

Compiler and Systems Infrastructures:

LLVM, ANTLR4

AI/ML:

PyTorch

Application And UI:

Flutter

EXPERIENCE

Data Science Lab, North South University – Research Assistant

September 2025 – December 2025

- Conducted research on k-clique listing algorithms, exploring modifications to the EBBkC algorithm.
- Explored a triangle based branching strategy to improve performance.
- Developed a **chordal fast path** to list maximal cliques in $O(n)$ for chordal candidate subgraphs and extract k-cliques from them.
- Built **experimental prototypes in C++** to evaluate performance and correctness on large graph datasets.

Projects:

GraphEasy – Ongoing Research Project

DSL for high performance graph analytics

GraphEasy is a **domain-specific language designed to accelerate graph analytics** by automatically parallelizing computation and optimizing data structures for high-performance execution on large graphs. It enables researchers to write high-level graph algorithms while the compiler handles **parallelism, vectorization, and memory optimization**.

Key Contributions:

- Designed and implemented a **full compiler pipeline**: ANTLR4 based frontend, Abstract Syntax Tree using C++, LLVM IR generator.
- Built an **auto-parallelising compiler** that identifies parallelizable loops using **program dependence graphs**, and generates parallel code using **pthreads**. Currently supports **doall** and **doacross** patterns.
- Developed **high-performance set operations** using **Roaring bitmaps**, leveraging **C++ vectorization (AVX2)** and **OpenMP** for union/intersection algorithms.
- Implemented memory arenas for dynamically growing sets, ensuring low-overhead memory management and fast access.
- Engineered LLVM passes to automatically transform and optimize code, integrating parallelization into the compiler.
- Optimized data structures and algorithms for scalable performance on large graphs using parallelization, SIMD, and memory-aware design.

Lattice

Distributed, LLVM-driven C++ Execution Platform

Designed and implemented a hardware-agnostic distributed execution platform that accepts unmodified C++ programs, extracts loop-level parallelism at the LLVM IR level, and runs iterations across volunteer worker machines with metadata-driven serialization and safe result merging.

Key Contributions:

- Built an LLVM-based compilation pipeline that emits IR, constructs a program dependence graph, and identifies do-all parallel loops using dependence analysis and Scalar Evolution.
- Implemented a custom LLVM transformation that rewrites eligible loops into per-iteration functions with explicitly captured execution state, enabling safe asynchronous execution and distribution.
- Designed a master-worker runtime using Drogon, C++ and gRPC that partitions iteration spaces, schedules work across remote workers, and dynamically manages worker availability and load.
- Engineered a metadata-driven serialization system to safely transmit captured program state across machines, reconstruct valid memory layouts

on workers, and prevent cross-address-space pointer errors.

- Implemented partition-aware result merging to correctly combine outputs from multiple workers without write conflicts or race conditions.

Self-Adapting Tiny Recursive Model

RL-Driven Model Self-Optimization

Developed a SEAL-inspired meta-learning system in which a reinforcement-learning meta-model dynamically adapts a compact recursive reasoning model by generating task-specific data augmentations and architectural configurations.

Key Contributions:

- Designed a two-model architecture consisting of a reinforcement-learning meta-model and a pretrained Tiny Recursive Model (TRM) for structured reasoning tasks.
- Implemented an outer-loop RL framework where the meta-model retrieves similar problems from a vector database, synthesizes task-specific training data, and proposes base-model hyperparameters and reasoning depth.
- Enabled dynamic architectural adaptation by training a minimal base model and conditionally attaching Low-Rank Adaptation (LoRA) modules to realize different reasoning depths without retraining full weights.
- Designed an automated inner-loop adaptation mechanism in which a pretrained base model is conditionally adapted by parameter-efficient LoRA updates, and post-adaptation performance is used as a reward signal for a reinforcement-learning meta-model.
- Explored compute-aware optimization by allowing the meta-model to trade off solution quality against architectural complexity. Work is ongoing on investigating the stability and efficacy of LoRA-based architecture swapping.

EDUCATION

North South University – Bsc in CSE

September 2022 – Present

Relevant Coursework: Graph Theory, Modeling and Simulations, Concept of Programming Language, Deep Learning, Natural Language Processing